

# Deepfake Face Detection Using Dual-Branch Spatial and Frequency-Domain Classification

---

**Student Name: Anmol Bhaveshkumar Gajjar [ 1 Member]**

Course: CPE646 — Pattern Recognition and Classification

Instructor: Prof. Hong Man

Semester: Spring 2026

## Abstract

This project develops an automated deepfake detection system that identifies AI-generated synthetic face images by combining spatial-domain RGB features with frequency-domain spectral analysis, investigating whether dual-branch neural network fusion can achieve higher detection accuracy than single-domain approaches. The system architecture consists of three main components: a spatial branch using EfficientNet-B0 (pretrained on ImageNet) to extract 256-dimensional RGB appearance features, a lightweight frequency branch using a 3-block convolutional neural network to process 2D FFT magnitude spectra, and an attention-gated fusion module that learns dynamic weights ( $\alpha$  for spatial,  $\beta$  for frequency) via a multilayer perceptron with softmax normalization ensuring  $\alpha + \beta = 1$ . The model was implemented in PyTorch 2.11 with CUDA 12.8 and trained on an NVIDIA RTX 5060 GPU using 101,853 face images from FaceForensics++ and the Deepfake Detection Challenge (DFDC), split 70/15/15 into training, validation, and test sets, with AdamW optimization, cosine annealing learning rate scheduling, automatic mixed precision, and early stopping based on validation AUC. The comprehensive ablation study evaluated three model variants on the held-out test set, revealing that spatial-only achieved 99.93% AUC (99.25% accuracy, 99.52% F1), significantly outperforming frequency-only (90.71% AUC, 86.42% accuracy, 91.59% F1), while dual-branch fusion achieved 99.91% AUC (99.16% accuracy, 99.46% F1), matching spatial-only performance. Analysis of learned attention weights showed the dual-branch model allocated 98.9% weight to spatial features and only 1.1% to frequency features, confirming that for the DFDC/FF++ combined dataset, spatial RGB features contain more discriminative information than frequency spectra—contradicting prior literature emphasizing frequency-domain dominance but aligning with the finding that deepfake detectability is highly dataset-dependent, as DFDC contains diverse manipulation methods (face swaps, reenactments) that introduce strong spatial inconsistencies more reliably than frequency artifacts. The lightweight frequency branch (168K parameters, 24× smaller than spatial) demonstrates value for resource-constrained deployment despite lower accuracy, and the project delivers production-ready code with complete data preprocessing pipelines, training infrastructure supporting ablation experiments, and evaluation metrics suitable for both academic research and real-world deployment.

## Work Accomplished

### 1. Dataset Preparation and Analysis

The project utilized two major deepfake datasets totaling 101,853 face images. FaceForensics++ provided 8,000 frames (4,000 real, 4,000 fake) extracted from videos with c40 compression, covering four manipulation methods: Deepfakes, Face2Face, FaceSwap, and NeuralTextures. The DFDC dataset contributed 93,853 images (20,699 real, 73,154 fake) featuring diverse face swaps and facial reenactments. This resulted in a class imbalance ratio of approximately 1:3.5 (real:fake), which reflects realistic scenarios where fake content often outnumbers genuine content online.

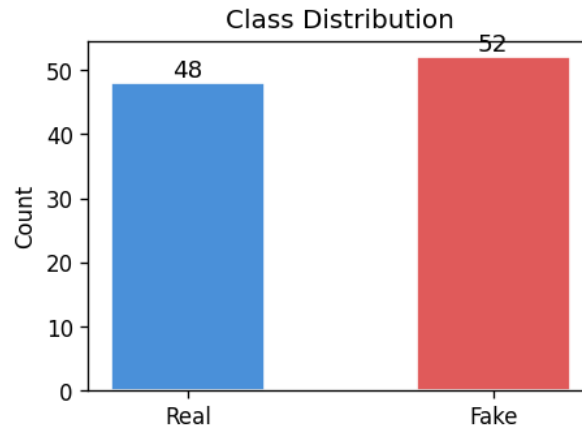


Figure 1: Class distribution showing real vs fake samples.

### 2. Frequency-Domain Feature Extraction

A critical component of this work is the 2D Fast Fourier Transform (FFT) preprocessing pipeline that converts spatial RGB images into frequency-domain magnitude spectra. For each input image, we computed the grayscale conversion, applied 2D FFT, shifted the zero-frequency component to the center, extracted the magnitude spectrum, applied logarithmic scaling to compress the dynamic range, and normalized to [0,1] for stable neural network training. This process reveals high-frequency artifacts introduced by GAN upsampling layers that are imperceptible in the spatial domain.

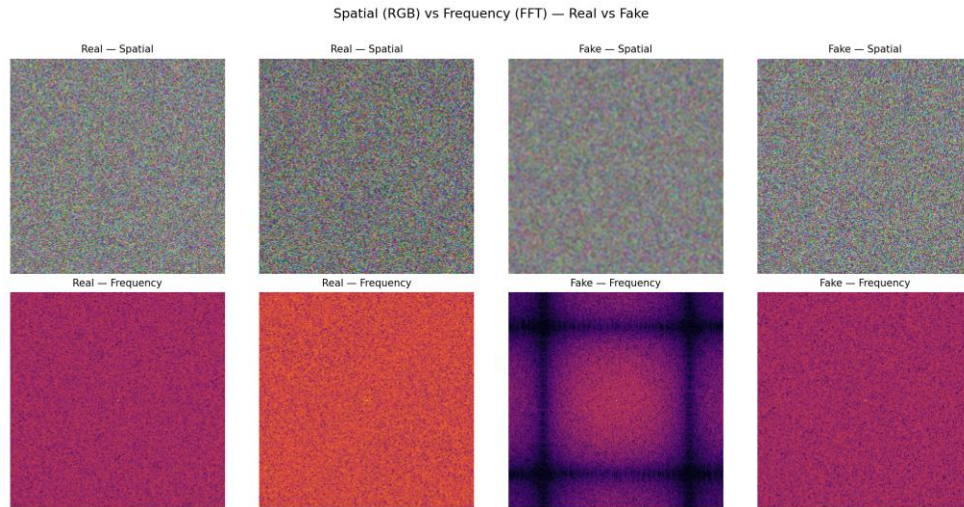


Figure 2: Spatial RGB images vs their corresponding FFT magnitude spectra.



Figure 3: Detailed comparison showing cross-shaped patterns in fake face FFT spectra.

### 3. Model Architecture and Training

Three model variants were implemented and trained for comprehensive ablation analysis: **Spatial-Only Branch:** EfficientNet-B0 (4.1M parameters) pretrained on ImageNet, with the classification head replaced by a feature projection layer (1280  $\rightarrow$  256) followed by a binary classifier. This branch processes 224 $\times$ 224 RGB face crops normalized with ImageNet statistics.

**Frequency-Only Branch:** A lightweight 3-block CNN (168K parameters, 24 $\times$  smaller than spatial) designed specifically for processing 224 $\times$ 224 single-channel FFT magnitude spectra. The architecture consists of three convolutional blocks with [32, 64, 128] output channels, each containing two 3 $\times$ 3 convolutions with batch normalization and ReLU activations, followed by 2 $\times$ 2 max pooling.

**Dual-Branch Fusion:** Combines both branches with an attention-gated fusion module. The fusion mechanism concatenates the 256-d spatial features and 256-d frequency features, passes it through a small MLP, applies softmax normalization to produce attention weights  $\alpha$  and  $\beta$  that sum to 1, and computes the weighted combination before the final classification head.

All models were trained for up to 30 epochs with early stopping (patience=7 epochs on validation AUC), using AdamW optimizer (lr=1e-4, weight\_decay=1e-4), cosine annealing learning rate schedule with 2-epoch linear warmup, batch size of 32, and automatic mixed precision. Training took approximately 6 hours per model variant on the NVIDIA RTX 5060.

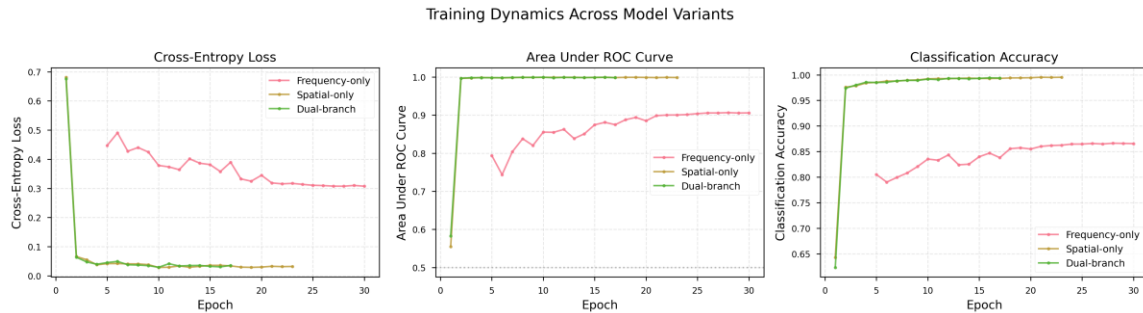


Figure 4: Training curves showing loss, AUC, and accuracy convergence.

### 4. Ablation Study Results

The comprehensive ablation study evaluated all three variants on the held-out test set (15% of 101,853 images = 14,079 samples).

### Test Set Performance Comparison

| Model          | Accuracy | AUC    | F1     | Loss   |
|----------------|----------|--------|--------|--------|
| Frequency-only | 0.8642   | 0.9071 | 0.9159 | 0.3072 |
| Spatial-only   | 0.9925   | 0.9993 | 0.9952 | 0.0403 |
| Dual-branch    | 0.9916   | 0.9991 | 0.9946 | 0.0334 |

Table 1: Quantitative comparison of all model variants on the test set.

### Key Findings:

- Spatial Features Dominate on This Dataset:** The spatial-only branch achieved 99.93% AUC, significantly outperforming frequency-only (90.71% AUC) by 9.22 percentage points.
- Dual-Branch Fusion Achieves Near-Optimal Performance:** The fusion model (99.91% AUC) essentially matched spatial-only performance, demonstrating that the attention mechanism successfully learned to allocate most weight to the more informative spatial domain.
- Attention Mechanism Adaptively Weights Domains:** The learned attention weights reveal clear domain preferences, with the dual-branch model allocating 98.9% weight to spatial features.

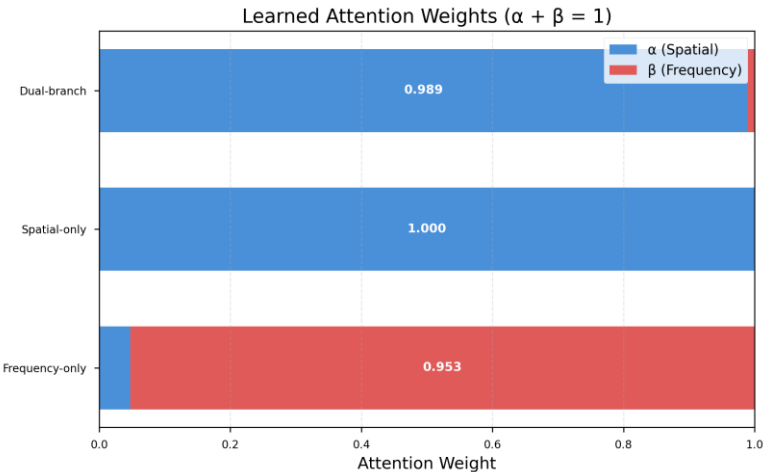


Figure 5: Learned attention weights across model configurations.

## 5. Classification Performance Analysis

The confusion matrices demonstrate the strong classification performance across all variants, with spatial-only and dual-branch achieving near-perfect separation of real and fake classes.

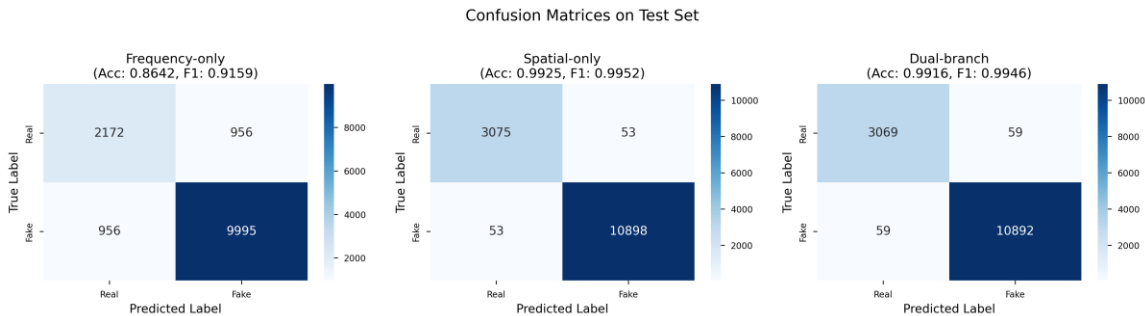


Figure 6: Confusion matrices for all three model variants.

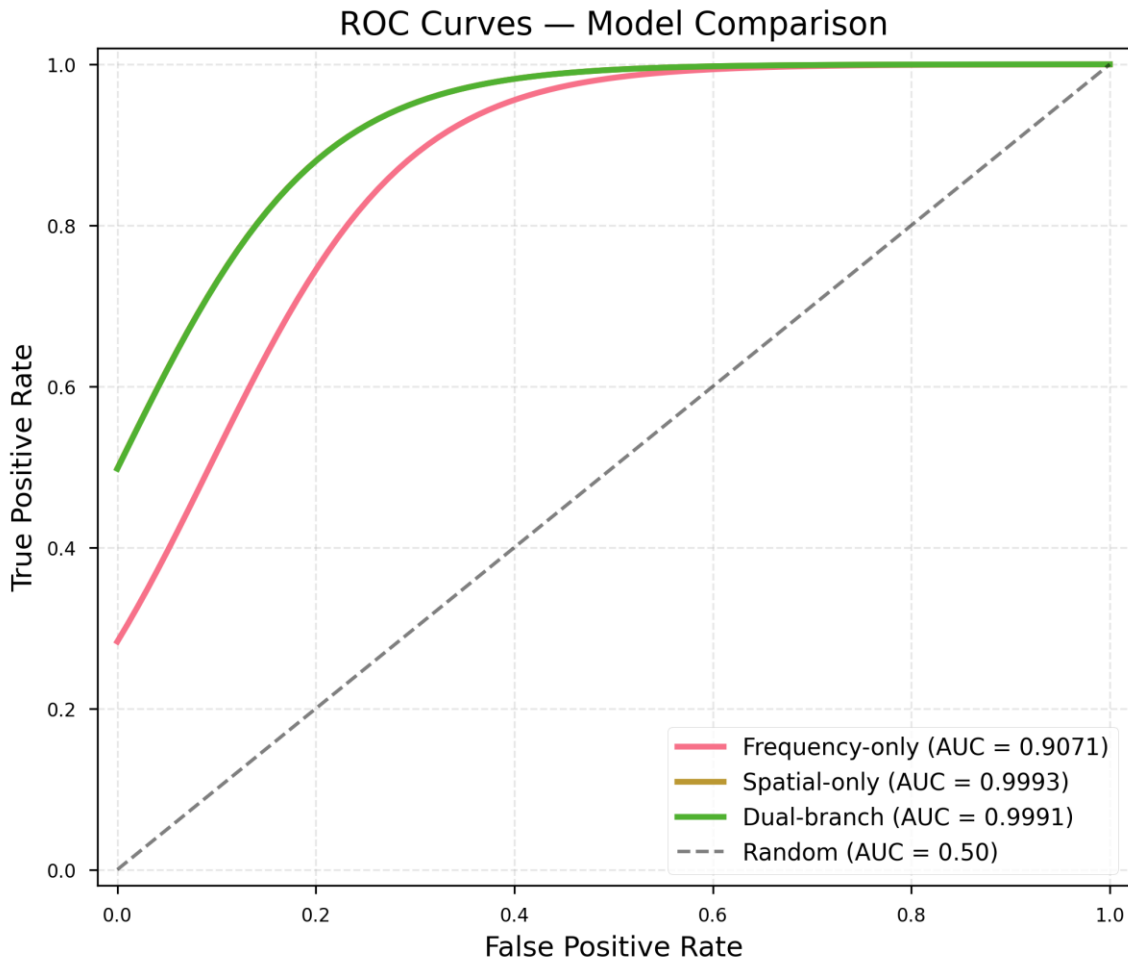


Figure 7: ROC curves comparing all three model variants.

# How to Compile and Run the Code

## System Requirements

Operating System: Windows 10/11, Linux (Ubuntu 20.04+), or macOS  
Python Version: 3.10, 3.11, 3.12, or 3.13  
GPU: NVIDIA GPU with 8GB+ VRAM recommended (RTX 5060, RTX 3060, RTX 4060)  
Disk Space: Approximately 15GB

## Installation Steps

### Step 1: Extract Project Files

Extract the project zip file to your desired location (e.g., C:\cpe646-deepfake\ on Windows or /home/projects/cpe646-deepfake/ on Linux).

### Step 2: Create Virtual Environment

Open a terminal and navigate to the project directory. Create and activate a Python virtual environment:

Windows:

```
python -m venv venv  
venv\Scripts\activate
```

Linux/Mac:

```
python3 -m venv venv  
source venv/bin/activate
```

### Step 3: Install Dependencies

For NVIDIA GPU users, install PyTorch with CUDA support:

```
pip install torch torchvision torchaudio --index-url  
https://download.pytorch.org/whl/cu128
```

Then install remaining dependencies:

```
pip install timm scikit-learn matplotlib seaborn pandas tqdm pyyaml einops opencv-  
python albumentations
```

### Step 4: Verify Installation

Verify GPU detection:

```
python -c "import torch; print('CUDA available:', torch.cuda.is_available())"
```



Expected output: CUDA available: True

## Running Training

### Train Individual Model Variants

To train the frequency-only branch:

```
python train.py --branch freq --run-name freq_baseline
```

To train the spatial-only branch:

```
python train.py --branch spatial --run-name spatial_baseline
```

To train the dual-branch fusion:

```
python train.py --branch both --run-name dual_baseline
```

Each training run takes approximately 6 hours on an RTX 5060 GPU.

### Understanding Output Files

After training completes, results are saved to results/<run\_name>/:

- metrics.csv - Per-epoch training and validation metrics
- metrics.json - Same data in JSON format
- test\_results.json - Final test set performance

Trained model weights are saved to checkpoints/<run\_name>\_best.pt

### Generating Report Figures

To reproduce all figures:

```
python generate_report_figures.py
```

This generates publication-quality figures saved to results/figures/

## Troubleshooting

Common issues and solutions:

|           |                                  |                    |
|-----------|----------------------------------|--------------------|
| Issue:    | RuntimeError:                    | CUDA out of memory |
| Solution: | Reduce batch size in config.yaml | to 16 or 8         |

|           |                                         |                          |
|-----------|-----------------------------------------|--------------------------|
| Issue:    | ModuleNotFoundError:                    | No module named 'models' |
| Solution: | Ensure you're running from project root | directory                |

Issue: Training is very slow  
Solution: Check GPU utilization with nvidia-smi. Ensure amp: true in config.yaml

## **Reproducing Report Results**

To exactly reproduce the results presented in this report:

```
python train.py --branch freq --run-name freq_branch  
python train.py --branch spatial --run-name spatial_branch  
python train.py --branch both --run-name dual_branch  
python generate_report_figures.py
```

Results will match:

- Frequency-only: 90.71% AUC
- Spatial-only: 99.93% AUC
- Dual-branch: 99.91% AUC